Conferencia 11: divide y vencerás, cuando no es evidente

Programación · Ciencia de la Computación y Ciencia de Datos · MatCom · 2026-11-25

En la conferencia pasada las tres partes de divide y vencerás se veían a simple vista. Para ordenar, partir es cortar la lista por la mitad, cada mitad es el mismo problema, y combinar es mezclar, o pegar. Nadie tuvo que preguntarse qué hacer con cada pedazo.

Hoy vemos tres problemas donde eso deja de ser obvio. En el primero partir sigue siendo fácil, pero combinar exige pensar un caso que no está en ninguna de las dos mitades, y para combinar barato hay que pedirle a cada mitad más de lo que el problema pide. En el segundo lo que no es obvio es qué partir. En el tercero hay que pedirle a cada mitad que, además de contar, ordene. Los tres tienen la misma pregunta detrás, y es la que hay que aprender a hacerse: **¿qué tengo que saber de cada mitad para poder combinar?**

1. La mejor racha

Un kiosco anota cada día cuánto ganó o perdió, en pesos. Estos son dieciséis días:

```
dias = [3, -5, 4, -1, 2, 6, -4, 2, 3, -2, 5, 1, -7, 4, -3, 2]
```

Queremos la **mejor racha**: los días seguidos, al menos uno, en los que el kiosco ganó más en total. Numeramos los días desde 0, como los índices de la lista. No vale saltarse días: si la racha va del día 2 al día 5, cuenta todo lo de en medio, sea ganancia o pérdida. En otros libros este problema se llama el **subarreglo de suma máxima**.

Si todos los números fueran positivos, la respuesta sería la lista entera. Los negativos lo complican: a veces vale la pena atravesar un día malo para llegar a días buenos, y a veces no.

Fuerza bruta

La solución obvia prueba todas las rachas: para cada día de inicio `i`, extiende la racha día por día y lleva la suma.

```
def mejor_racha_bruta(v):
    """La mayor suma de un pedazo de días seguidos de v, probando todos."""
    mejor = v[0]
    for i in range(len(v)):
        suma = 0
        for j in range(i, len(v)):
            suma += v[j]              # suma de v[i..j]
            if suma > mejor:
                mejor = suma
    return mejor

print(mejor_racha_bruta(dias))
```

```
16
```

Hay un ciclo dentro de otro, y el de dentro da `n - i` vueltas: en total $\frac{n(n+1)}{2}$, el mismo orden n^2 que la selección de la conferencia 8. Funciona, y es el oráculo contra el que vamos a probar todo lo demás.

2. Partir por la mitad

Probemos divide y vencerás. Partimos los días por la mitad y buscamos la mejor racha de cada mitad, recursivamente. ¿La mejor racha de todo es la mejor de las dos?

No necesariamente. La mejor racha puede **cruzar** el punto de corte: empezar en la mitad izquierda y terminar en la derecha. Esa racha no es una racha de ninguna de las dos mitades, así que ninguna de las dos llamadas recursivas la va a encontrar. Así que hay tres candidatas:

1. La mejor racha que está entera en la mitad izquierda: la da una llamada recursiva.
2. La mejor racha que está entera en la mitad derecha: la da la otra.
3. La mejor racha que cruza el medio: hay que calcularla aparte.

La tercera parece tan difícil como el problema original, pero no lo es. Una racha que cruza el medio está formada por un pedazo que **termina** justo antes del medio y un pedazo que **empieza** justo en el medio, y los dos se pueden escoger por separado. La mejor racha que cruza es la mejor de las que terminan en el medio, pegada a la mejor de las que empiezan en el medio. Y cada una de esas es un solo ciclo: desde el medio hacia la izquierda, llevando la suma, y desde el medio hacia la derecha.

```
def mejor_cruce(v, izq, medio, der):
    """La mejor suma de un pedazo de v[izq:der] que cruza el medio."""
    suma, mejor_izq = 0, v[medio - 1]
    for i in range(medio - 1, izq - 1, -1):     # del medio hacia la izquierda
        suma += v[i]
        mejor_izq = max(mejor_izq, suma)

    suma, mejor_der = 0, v[medio]
    for j in range(medio, der):                 # del medio hacia la derecha
        suma += v[j]
        mejor_der = max(mejor_der, suma)

    return mejor_izq + mejor_der

def mejor_racha(v, izq=0, der=None):
    """La mayor suma de un pedazo de días seguidos de v[izq:der]."""
    if der is None:
        der = len(v)
    if der - izq == 1:
        return v[izq]                           # un solo día
    medio = (izq + der) // 2
    return max(mejor_racha(v, izq, medio),
               mejor_racha(v, medio, der),
               mejor_cruce(v, izq, medio, der))

print(mejor_racha(dias))
```

```
16
```

En lugar de rebanadas usamos los índices `izq` y `der`, como en la búsqueda binaria recursiva de la conferencia 9, con `der` fuera del pedazo, igual que en `range` y en las rebanadas. Así ninguna llamada copia la lista. El `range(medio - 1, izq - 1, -1)` cuenta hacia atrás: de `medio - 1` hasta `izq`, porque el final de un `range` no se incluye.

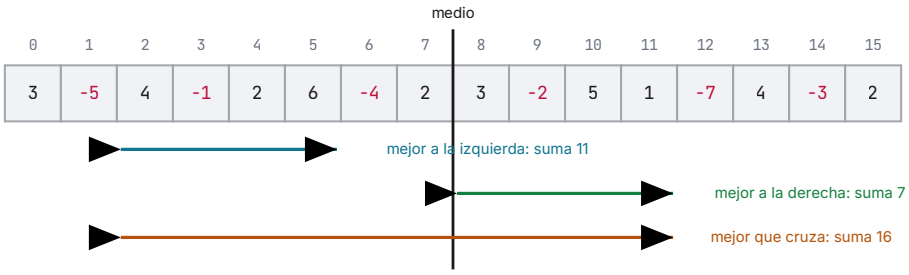


Figura 1. Las tres candidatas en el primer nivel de la recursión. La que cruza el medio no la encuentra ninguna de las dos llamadas recursivas, y en este ejemplo es la que gana.

En la figura 1 la mejor racha de la izquierda suma 11 y la de la derecha 7, pero la que cruza suma 16: atraviesa el día malo del índice 6 para juntar los días buenos de los dos lados. Si solo hubiéramos combinado las dos respuestas de las mitades, el programa habría devuelto 11 sin ningún error.

Probamos contra la fuerza bruta, con muchas listas pequeñas que tengan negativos:

```
random.seed(1)
for prueba in range(2000):
    v = [random.randint(-10, 10) for _ in range(random.randint(1, 12))]
    if mejor_racha(v) != mejor_racha_bruta(v):
        print("FALLA", v)
print("pruebas terminadas")
```

```
pruebas terminadas
```

Cuánto cuesta

Es el árbol de la ordenación por mezcla. Cada llamada parte su pedazo a la mitad, así que hay $\log_2 n$ niveles, y en cada nivel los cruces recorren, entre todos, cada día una vez: n de trabajo por nivel. Total, $n \log_2 n$, igual que la mezcla. Aquí `mejor_cruce` hace el papel de `mezclar`.

3. Devolver más de lo que se pide

Se puede hacer mejor, y la manera de hacerlo es la idea más importante de la clase.

`mejor_cruce` recorre el pedazo entero porque necesita dos cosas que las llamadas recursivas no le dieron: la mejor suma que **termina** en el borde derecho de la mitad izquierda, y la mejor que **empieza** en el borde izquierdo de la mitad derecha. Las llamadas recursivas recorrieron esos mismos días, y podrían haber calculado esas sumas por el camino, pero solo devolvían la mejor racha.

Así que hacemos que cada llamada devuelva **cuatro** números sobre su pedazo, en lugar de uno:

- `total`, la suma de todo el pedazo;
- `prefijo`, la mejor suma de un pedazo que empieza en el borde izquierdo;
- `sufijo`, la mejor suma de un pedazo que termina en el borde derecho;
- `mejor`, la mejor racha en cualquier lugar, que es la respuesta.

La pregunta es si los cuatro números del pedazo entero se pueden calcular a partir de los cuatro de cada mitad, sin mirar los días. Se puede, y cada uno se razona en una línea:

- El `total` es la suma de los dos totales.

- El mejor `prefijo` del todo o se queda en la mitad izquierda, y entonces es el prefijo de la izquierda, o la atraviesa entera y sigue en la derecha: el total de la izquierda más el prefijo de la derecha.
- El `sufijo` es lo mismo al revés.
- La `mejor` racha está en la izquierda, en la derecha, o cruza. Y la que cruza es el sufijo de la izquierda más el prefijo de la derecha, que ya los tenemos.

```
def resumen(v, izq, der):
    """Devuelve (total, prefijo, sufijo, mejor) de v[izq:der]."""
    if der - izq == 1:
        x = v[izq]
        return x, x, x, x
    medio = (izq + der) // 2
    t1, p1, s1, m1 = resumen(v, izq, medio)
    t2, p2, s2, m2 = resumen(v, medio, der)
    total = t1 + t2
    prefijo = max(p1, t1 + p2)
    sufijo = max(s2, t2 + s1)
    mejor = max(m1, m2, s1 + p2)
    return total, prefijo, sufijo, mejor

def mejor_racha_rapida(v):
    return resumen(v, 0, len(v))[3]

print(resumen(dias, 0, len(dias)))

random.seed(2)
for prueba in range(2000):
    v = [random.randint(-10, 10) for _ in range(random.randint(1, 12))]
    if mejor_racha_rapida(v) != mejor_racha_bruta(v):
        print("FALLA", v)
print("pruebas terminadas")
```

```
(10, 14, 12, 16)
pruebas terminadas
```

Ahora combinar cuesta lo mismo tenga el pedazo dos días o un millón: unas pocas sumas y `max`. El árbol tiene n hojas, una por día, y $n - 1$ nodos internos: cada nodo interno junta dos pedazos en uno, y para pasar de n pedazos a uno solo hacen falta $n - 1$ juntas. Así que hay $2n - 1$ llamadas, cada una con trabajo constante. El total es proporcional a n, mejor que $n \log n$.

Medimos las tres versiones:

```
def cronometrar(funcion, v):
    inicio = time.perf_counter()
    funcion(v)
    return time.perf_counter() - inicio

random.seed(8)
print(f"{'n':>5} {'bruta':>9} {'mitades':>9} {'resumen':>9}")
for n in [500, 1000, 2000, 4000]:
    v = [random.randint(-100, 100) for _ in range(n)]
    print(f"{n:>5} {cronometrar(mejor_racha_bruta, v):>8.3f}s "
          f"{cronometrar(mejor_racha, v):>8.3f}s "
          f"{cronometrar(mejor_racha_rapida, v):>8.3f}s")
```

```
    n     bruta   mitades   resumen
  500    0.010s    0.001s    0.000s
 1000    0.028s    0.001s    0.000s
```

```
 2000    0.116s    0.003s    0.002s
 4000    0.457s    0.006s    0.003s
```

Con estos tamaños la fuerza bruta ya está lejos, pero las dos versiones de divide y vencerás tardan tan poco que no se distinguen. Para ver la diferencia entre $n \log n$ y n hacen falta listas más grandes, sin la fuerza bruta:

```python
random.seed(8)
print(f"{'n':>7} {'mitades':>9} {'resumen':>9}")
for n in [10_000, 20_000, 40_000, 80_000]:
    v = [random.randint(-100, 100) for _ in range(n)]
    print(f"{n:>7} {cronometrar(mejor_racha, v):>8.4f}s "
          f"{cronometrar(mejor_racha_rapida, v):>8.4f}s")
```

```
      n   mitades   resumen
  10000   0.0173s   0.0061s
  20000   0.0373s   0.0112s
  40000   0.0804s   0.0226s
  80000   0.1664s   0.0441s
```

Lee las columnas de arriba abajo. Las dos se duplican, más o menos, con cada duplicación de n, porque $\log_2 n$ crece tan despacio que entre 10 000 y 80 000 solo pasa de 13 a 16. La diferencia está en la constante: `mejor_racha` recorre el pedazo entero en cada llamada interna, y `resumen` solo hace unas sumas.

Lo que cambió entre `mejor_racha` y `resumen` no es la división: las dos parten igual. Es lo que devuelve cada llamada. Pedirle a cada mitad un poco más de lo que el problema pide, el total, el prefijo y el sufijo, es lo que deja combinar sin volver a mirar los datos. Es un truco que se repite: cuando combinar es caro, pregúntate qué información sobre cada mitad lo haría barato, y haz que la recursión la devuelva.

4. La potencia rápida

El segundo problema parece no tener nada que partir. Queremos calcular a^n. La manera obvia hace n multiplicaciones:

```python
def potencia_lenta(a, n):
    resultado = 1
    for _ in range(n):
        resultado *= a
    return resultado

print(potencia_lenta(3, 13))
```

```
1594323
```

No hay ninguna lista aquí. Lo que se parte es el **exponente**. Si n es par, $a^n = a^{n/2} \cdot a^{n/2}$, y basta calcular $a^{n/2}$ una vez y multiplicarlo por sí mismo. Si n es impar, $a^n = a^{(n-1)/2} \cdot a^{(n-1)/2} \cdot a$. En Python, en los dos casos la mitad es `n // 2`:

```python
def potencia(a, n):
    if n == 0:
        return 1
    mitad = potencia(a, n // 2)
    if n % 2 == 0:
        return mitad * mitad
```

```
    return mitad * mitad * a

print(potencia(3, 13))
print(all(potencia(a, n) == a ** n for a in range(-3, 6) for n in range(40)))
```

```
1594323
True
```

El `all(...)` de la última línea pregunta si todas las comparaciones son ciertas, para 9 bases y 40 exponentes. Cada llamada divide el exponente entre dos, así que hay unos $\log_2 n$ niveles, cada uno con una o dos multiplicaciones. Contamos:

```
multiplicaciones = 0

def potencia_contando(a, n):
    global multiplicaciones
    if n == 0:
        return 1
    mitad = potencia_contando(a, n // 2)
    if n % 2 == 0:
        multiplicaciones += 1
        return mitad * mitad
    multiplicaciones += 2
    return mitad * mitad * a

for n in [10, 1000, 1_000_000, 10 ** 18]:
    multiplicaciones = 0
    potencia_contando(1, n)
    print(f"n = {n:>19}: {multiplicaciones:>3} multiplicaciones en lugar de {n}")
```

```
n =                  10:   6 multiplicaciones en lugar de 10
n =                1000:  16 multiplicaciones en lugar de 1000
n =             1000000:  27 multiplicaciones en lugar de 1000000
n = 1000000000000000000:  84 multiplicaciones en lugar de 1000000000000000000
```

Usamos la base 1 para que el resultado no crezca y la cuenta se haga en un instante. Para $n = 10^{18}$ la potencia lenta necesitaría un trillón de multiplicaciones; la rápida, menos de cien.

Un error que deshace todo

Esta versión parece la misma idea y cambia una sola línea: en lugar de guardar la mitad en una variable, la calcula donde la necesita.

```
def potencia_mal(a, n):
    global multiplicaciones
    if n == 0:
        return 1
    if n % 2 == 0:
        multiplicaciones += 1
        return potencia_mal(a, n // 2) * potencia_mal(a, n // 2)
    multiplicaciones += 2
    return potencia_mal(a, n // 2) * potencia_mal(a, n // 2) * a

for n in [16, 256, 4096]:
    multiplicaciones = 0
    potencia_mal(1, n)
    print(f"n = {n:>4}: {multiplicaciones} multiplicaciones")
```

```
n =   16: 47 multiplicaciones
n =  256: 767 multiplicaciones
n = 4096: 12287 multiplicaciones
```

Da el resultado correcto y hace **más** multiplicaciones que la potencia lenta. Ahora cada llamada hace dos llamadas, y el árbol tiene unos $\log_2 n$ niveles, cada uno con el doble de llamadas que el anterior: $1 + 2 + 4 + \cdots$ hasta pasar de n. La tabla dice que salen unas tres multiplicaciones por cada unidad del exponente. Es el árbol de Fibonacci de la conferencia 9, que calculaba lo mismo varias veces. La potencia rápida es rápida porque resuelve **un** subproblema y lo usa dos veces, como la búsqueda binaria, que también tenía un solo subproblema. Dividir no ayuda si después resuelves los pedazos repetidos.

Para qué sirve

Con números de Python, que no tienen límite de tamaño, `a ** n` ya usa esta idea por dentro. Donde se nota es cuando solo interesa el resto de dividir entre algo, por ejemplo los últimos tres dígitos de $7^{10^{18}}$. Basta tomar el resto en cada multiplicación, para que los números nunca crezcan:

```python
def potencia_resto(a, n, m):
    """a ** n % m, sin calcular a ** n."""
    if n == 0:
        return 1 % m          # y no 1: si m es 1, todo resto es 0
    mitad = potencia_resto(a, n // 2, m)
    if n % 2 == 0:
        return mitad * mitad % m
    return mitad * mitad * a % m

print(potencia_resto(7, 10 ** 18, 1000))
print(potencia_resto(7, 12345, 1000) == 7 ** 12345 % 1000)
```

```
1
True
```

El primer resultado es 1: $7^{10^{18}}$ termina en `001`. No es casualidad: 7^{20} ya termina en `001`, y como 10^{18} es múltiplo de 20, $7^{10^{18}}$ es una potencia de 7^{20}, y un número que termina en `001` multiplicado por sí mismo sigue terminando en `001`. La segunda línea comprueba la función contra `**` con un exponente que todavía se puede calcular entero.

Es el cálculo central de RSA, uno de los sistemas de criptografía de clave pública con los que se construyó `https`, que usa exponentes y restos de cientos de cifras. Los sistemas más usados hoy en `https` trabajan con curvas elípticas en lugar de restos, pero con la misma idea de duplicar y sumar. Con la potencia lenta no terminaría nunca.

5. Contar inversiones

El tercer problema: ¿cuán desordenada está una lista? Una medida natural es contar las **inversiones**, los pares de posiciones `i < j` donde `v[i] > v[j]`, es decir, los pares que están al revés. Una lista ordenada tiene 0 inversiones. Una lista al revés tiene todas las posibles, $\frac{n(n-1)}{2}$.

Sirve, por ejemplo, para comparar dos clasificaciones. Si dos personas ordenan las mismas diez canciones según su gusto, se numeran las canciones según la primera persona, se escriben en el orden de la segunda, y las inversiones cuentan los pares de canciones en que no están de acuerdo.

La fuerza bruta mira todos los pares:

```python
def inversiones_bruta(v):
    total = 0
    for i in range(len(v)):
        for j in range(i + 1, len(v)):
            if v[i] > v[j]:
                total += 1
    return total

print(inversiones_bruta([3, 1, 4, 1, 5, 9, 2, 6]))
```

```
8
```

Con divide y vencerás, partimos por la mitad. Las inversiones son de tres clases: las que tienen los dos elementos en la izquierda, las que los tienen en la derecha, y las que **cruzan**, con uno en cada mitad. Las dos primeras las cuentan las llamadas recursivas. Las que cruzan, otra vez, son el problema.

Contarlas par por par es n^2 otra vez. Pero fíjate en una cosa: para las inversiones que cruzan, el orden **dentro** de cada mitad no importa. Solo importa qué elemento de la izquierda es mayor que qué elemento de la derecha. Así que podemos pedirle a cada llamada recursiva que, además de contar, **devuelva su mitad ordenada**. Y con dos mitades ordenadas, las inversiones que cruzan se cuentan mientras se mezclan.

Mira la mezcla de la conferencia 10. Cuando el elemento que se toma viene de la derecha, `b[j]`, es porque es menor que `a[i]`, y como `a` está ordenada, también es menor que todos los que quedan en `a` después de `a[i]`. Cada uno de esos forma una inversión con `b[j]`. Son `len(a) - i`, y se suman de una vez:

```python
def ordenar_contando(v):
    """Devuelve (v ordenada, cantidad de inversiones de v)."""
    if len(v) <= 1:
        return v, 0
    medio = len(v) // 2
    a, inv_a = ordenar_contando(v[:medio])
    b, inv_b = ordenar_contando(v[medio:])

    mezcla, cruzan = [], 0
    i, j = 0, 0
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            mezcla.append(a[i])
            i += 1
        else:
            mezcla.append(b[j])
            cruzan += len(a) - i      # b[j] es menor que a[i], a[i+1], ...
            j += 1
    mezcla.extend(a[i:])
    mezcla.extend(b[j:])
    return mezcla, inv_a + inv_b + cruzan

print(ordenar_contando([3, 1, 4, 1, 5, 9, 2, 6]))

random.seed(3)
for prueba in range(2000):
    v = [random.randint(1, 6) for _ in range(random.randint(0, 12))]
    if ordenar_contando(v)[1] != inversiones_bruta(v):
        print("FALLA", v)
print("pruebas terminadas")
```

```
([1, 1, 2, 3, 4, 5, 6, 9], 8)
pruebas terminadas
```

El $\leqslant$ importa otra vez. Con dos elementos iguales se toma primero el de la izquierda, y no se cuenta inversión, que es lo correcto: dos iguales no están al revés. Con <, cada par de iguales que cruza contaría como inversión, y las pruebas, que usan números del 1 al 6 para que haya muchos repetidos, lo atraparían.

La función es la ordenación por mezcla con un contador: $n \log_2 n$. Contra la fuerza bruta:

```
random.seed(8)
print(f"{'n':>5} {'bruta':>9} {'mezcla':>9}")
for n in [500, 1000, 2000, 4000]:
    v = [random.randint(1, 1_000_000) for _ in range(n)]
    print(f"{n:>5} {cronometrar(inversiones_bruta, v):>8.3f}s "
          f"{cronometrar(ordenar_contando, v):>8.3f}s")
```

```
    n     bruta    mezcla
  500    0.006s    0.001s
 1000    0.026s    0.002s
 2000    0.098s    0.004s
 4000    0.385s    0.008s
```

Fíjate en que la versión rápida resuelve un problema **más grande** que el que se le pidió: cuenta y además ordena. Es lo mismo que pasó con la mejor racha. Ordenar no era parte del problema, pero sin la lista ordenada no había manera barata de contar lo que cruza.

6. La pregunta

Los cinco problemas de las dos últimas clases, puestos juntos:

PROBLEMA	DIVIDIR	QUÉ DEVUELVE CADA MITAD	COMBINAR
Ordenar por mezcla	por la mitad, gratis	la mitad ordenada	mezclar, n
Ordenación rápida	alrededor de un pivote, n	la parte ordenada	pegar, gratis
Mejor racha	por la mitad	total, prefijo, sufijo y mejor	unas sumas y `max`
Potencia	el exponente a la mitad	$a^{n/2}$, una sola vez	una o dos multiplicaciones
Inversiones	por la mitad	la mitad ordenada y su cuenta	mezclar y contar, n

La columna del medio es la que hay que inventar, y la pregunta que lleva a ella es siempre la misma: **¿qué necesito saber de cada mitad para poder combinar sin volver a mirar todo?** Si la respuesta es "solo la respuesta del problema", el diseño es directo. Si no, se le pide más a la recursión, y casi siempre ese más es poco: una lista ordenada, una suma, un prefijo.

Y hay una advertencia que la potencia dejó clara: el costo lo decide el árbol de llamadas. Contar cuántos niveles tiene y cuánto trabajo hay en cada uno da la respuesta, y si un subproblema se resuelve dos veces, el árbol se llena de trabajo repetido.

7. Resumen

- En divide y vencerás, partir puede ser fácil y combinar no. La mejor racha puede **cruzar** el medio, y ninguna llamada recursiva la ve.

- La mejor racha que cruza es la mejor que termina en el medio más la mejor que empieza en el medio, y se calcula con dos ciclos: $n \log_2 n$ en total.
- Si cada llamada devuelve también el total, el mejor prefijo y el mejor sufijo, combinar cuesta lo mismo con cualquier tamaño y el total baja a n.
- La **potencia rápida** parte el exponente: a^n con unas $2 \log_2 n$ multiplicaciones. Calcular la mitad dos veces en lugar de guardarla lleva el costo de vuelta a n.
- Una **inversión** es un par de posiciones que están al revés. Se cuentan en $n \log_2 n$ si cada mitad devuelve su lista ordenada, contando mientras se mezcla.
- La pregunta que guía el diseño: qué hay que saber de cada mitad para combinar. A veces la respuesta es resolver un problema más grande que el pedido.
- Todo algoritmo de divide y vencerás se prueba contra la fuerza bruta, con muchas entradas pequeñas al azar.

Ejercicios

1. Modifica `mejor_racha_bruta` y `resumen` para que devuelvan, además de la suma, el primer y el último día de la mejor racha. ¿Qué más tiene que devolver cada llamada de `resumen`?
2. ¿Qué devuelve `mejor_racha` si todos los días son pérdidas? ¿Y si se permitiera la racha vacía, de cero días? Cambia las funciones para ese caso y prueba las dos contra una fuerza bruta escrita para la nueva regla.
3. Existe una solución de la mejor racha con **un solo ciclo** que no es divide y vencerás: recorrer los días llevando la mejor suma de una racha que termine en el día actual. Escríbela, pruébala contra la fuerza bruta y mídela contra `resumen`. Pista: la mejor racha que termina hoy es la de hoy sola, o la mejor que terminaba ayer más hoy.
4. Escribe `potencia` sin recursión, mirando los bits del exponente: $a^{13} = a^8 \cdot a^4 \cdot a^1$, porque 13 es `1101` en binario. Lleva una variable que vaya valiendo $a, a^2, a^4, a^8, \ldots$
5. Usa `potencia_resto` para encontrar los últimos cuatro dígitos de $2^{2^{100}}$. Antes de ejecutarlo, estima cuántas multiplicaciones hace.
6. Dos personas ordenan las canciones `A` a `J` así: la primera `ABCDEFGHIJ`, la segunda `CABDFEJGIH`. Usa `ordenar_contando` para medir en cuántos pares no están de acuerdo. ¿Cuál es el máximo posible con diez canciones?
7. El **par más cercano** en una recta: dada una lista de números, encuentra los dos más cercanos entre sí. Resuélvelo por fuerza bruta, y después con divide y vencerás: parte la lista **ordenada** por la mitad. ¿Qué par puede cruzar el medio? ¿Cuántos candidatos tiene? ¿Hacía falta divide y vencerás, una vez que la lista está ordenada?
8. Escribe `mayoritario(v)` que diga si algún valor aparece en más de la mitad de las posiciones, y cuál. Con divide y vencerás: si un valor es mayoritario en `v`, tiene que ser mayoritario en al menos una de las dos mitades. ¿Por qué? ¿Qué devuelve cada mitad, y cuánto cuesta combinar?